

0.0.1 Question 1c

Discuss one thing you notice that is different between the two emails that might relate to the identification of spam.

One thing I notice that is different between the two emails is that in the ham email, the user can actually click on the links in the email whereas in the spam email, you cannot click on the links (and the links also seem invalid). Additionally, within the spam email's text, there is a lot of encoding/embedding characters whereas the ham email you can read the message without seeing any of those.

0.0.2 Question 3a

Create a bar chart like the one above comparing the proportion of spam and ham emails containing certain words. Choose a set of words that are different from the ones above, but also have different proportions for the two classes. Make sure to only consider emails from `train`.

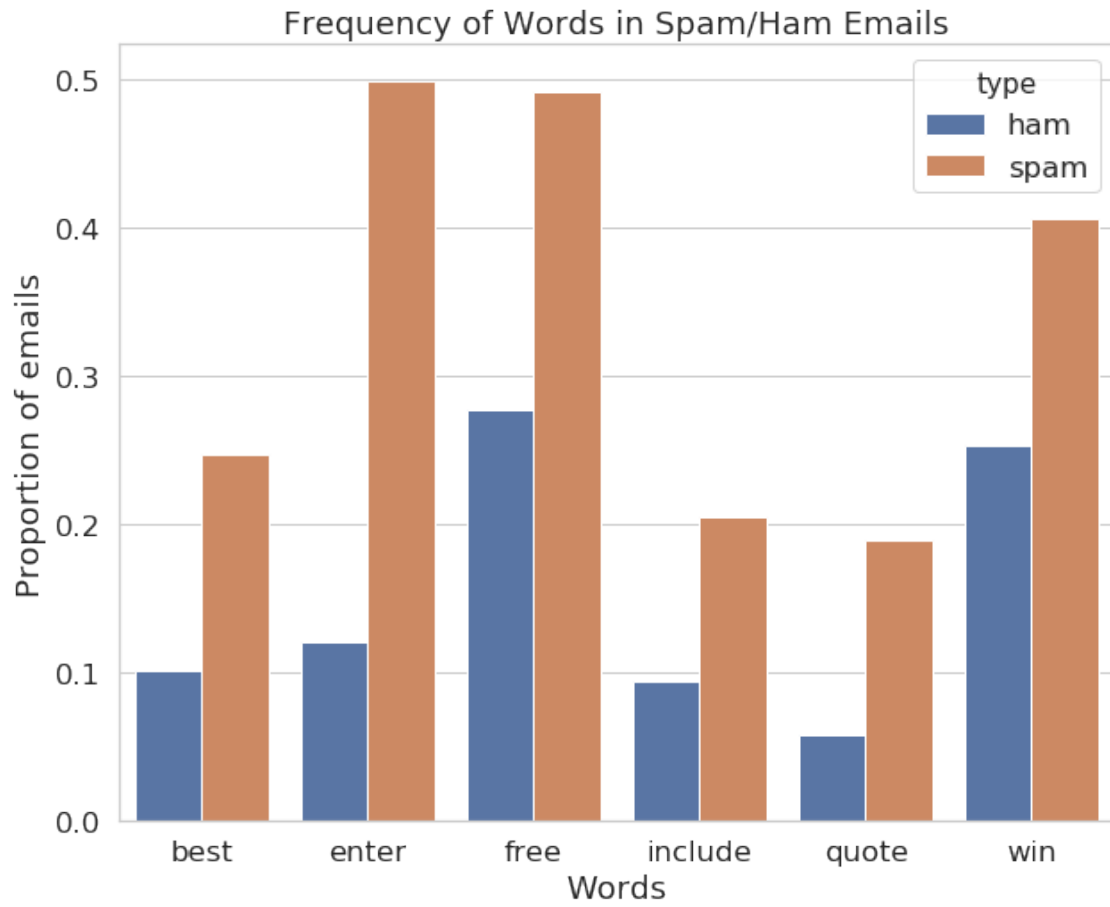
```
In [13]: train=train.reset_index(drop=True) # We must do this in order to preserve the ordering of email
val=val.reset_index(drop=True)
test=test.reset_index(drop=True)

# Number of ham and spam emails
num_ham = len(train[train['spam'] == 0])
num_spam = len(train[train['spam'] == 1])

# TURN THIS QUESTION INTO A FUNCTION
# CHOOSE WORDS TO USE AND PLOTS PROPORTIONS OF EACH WORD
def word_proportions(words_to_use):
    words_in_texts_array = words_in_texts(words_to_use, train['email'])
    # Creating a new dataframe of the chosen words as the variables, melting the table, adding
    new_df = pd.DataFrame(words_in_texts_array, columns = words_to_use)
    new_df['type'] = train['spam'].apply(lambda x: "ham" if x == 0 else "spam")
    new_df = new_df.melt(id_vars=['type'], value_vars=words_to_use)
    new_df['total'] = new_df['type'].apply(lambda x: num_ham if x == "ham" else num_spam)
    new_df['proportion'] = new_df['value']/new_df['total']
    train_df = new_df[['type', 'variable', 'proportion']]
    train_df = train_df.groupby(['type', 'variable']).sum().reset_index()
    plt.figure(figsize=(10,8))
    sns.barplot(data=train_df, x='variable', y='proportion', hue='type')
    plt.title("Frequency of Words in Spam/Ham Emails")
    plt.xlabel("Words")
    plt.ylabel("Proportion of emails")

words_to_use = ['best', 'win', 'enter', 'quote', 'free', 'include']

word_proportions(words_to_use)
```



0.0.3 Question 3b

Create a *class conditional density plot* like the one above (using `sns.distplot`), comparing the distribution of the length of spam emails to the distribution of the length of ham emails in the training set. Set the x-axis limit from 0 to 50000.

```
In [14]: len_email_spam = train.query("spam == 1")
len_email_spam['length'] = len_email_spam['email'].apply(len)
len_email_ham = train.query("spam == 0")
len_email_ham['length'] = len_email_ham['email'].apply(len)
sns.distplot(len_email_ham['length'], hist=False, label="Ham")
sns.distplot(len_email_spam['length'], hist=False, label="Spam")
plt.xlim(0, 50000)
plt.title("Distribution of email body's length")
plt.xlabel("Length of email body")
plt.ylabel("Distribution")
```

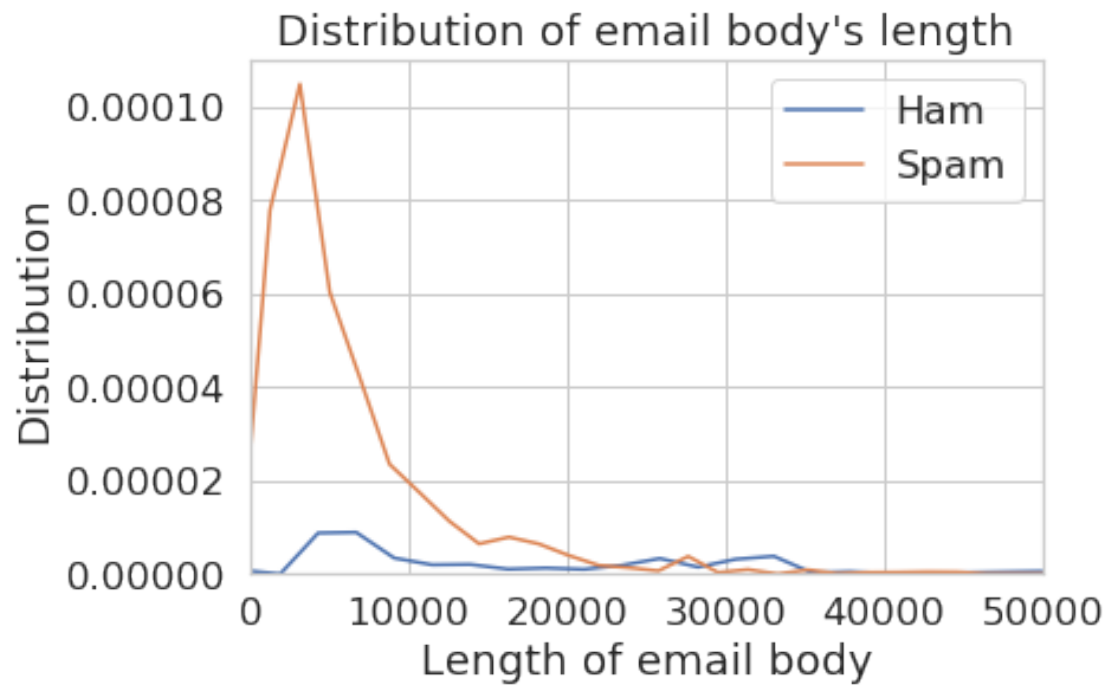
```
/srv/conda/envs/data100/lib/python3.7/site-packages/ipykernel_launcher.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: [http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.h](http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html)

```
/srv/conda/envs/data100/lib/python3.7/site-packages/ipykernel_launcher.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: [http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.h](http://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html)
after removing the cwd from sys.path.

```
Out[14]: Text(0, 0.5, 'Distribution')
```



0.0.4 Question 6c

Provide brief explanations of the results from 6a and 6b. Why do we observe each of these values (FP, FN, accuracy, recall)?

1. We have 0 false positives because our predictor always predicts negative, never positive.
2. We have false negatives when we predict negative, but the actual outcome is positive. Because it is always predicting negative, the number of false negatives is just the number of actual positives.
3. Accuracy is the total number of predictions correct out of all the emails; thus it is the number of actual negatives over total emails since our predictor always predicts negative.
4. The recall ($\# \text{ true positives} / \# \text{ actual positives}$) is 0 because our predictor always predicts negative, never positive so we have 0 true positives.

0.0.5 Question 6e

Are there more false positives or false negatives when using the logistic regression classifier from Question 5?

When using the logistic regression classifier, there are more false positives than false negatives.

0.0.6 Question 6f

1. Our logistic regression classifier got 75.6% prediction accuracy (number of correct predictions / total). How does this compare with predicting 0 for every email?
 2. Given the word features we gave you above, name one reason this classifier is performing poorly. Hint: Think about how prevalent these words are in the email set.
 3. Which of these two classifiers would you prefer for a spam filter and why? Describe your reasoning and relate it to at least one of the evaluation metrics you have computed so far.
-
1. By predicting 0 for every email, we got 74.5% prediction accuracy. This is surprising really close to our regression classifier prediction accuracy; it is only off by 1%.
 2. This classifier is performing poorly because the words ['drug', 'bank', 'prescription', 'memo', 'private'] can be used in both spam and ham emails! Although used more often in ham emails, these features do not do a decent enough job of distinguishing spam emails from ham emails.
 3. I would prefer the logistic regression classifier because I can always refit my model after selecting better features. If I could not refit my model, I would still choose the logistic regression classifier because it has a higher recall and prediction accuracy than just simply predicting 0 for every email. Additionally, there is NO point in building a ham/spam classifier if the predictor is just going to predict ham for every email.

0.0.7 Question 7: Feature/Model Selection Process

In this following cell, describe the process of improving your model. You should use at least 2-3 sentences each to address the follow questions:

1. How did you find better features for your model?
2. What did you try that worked / didn't work?
3. What was surprising in your search for good features?

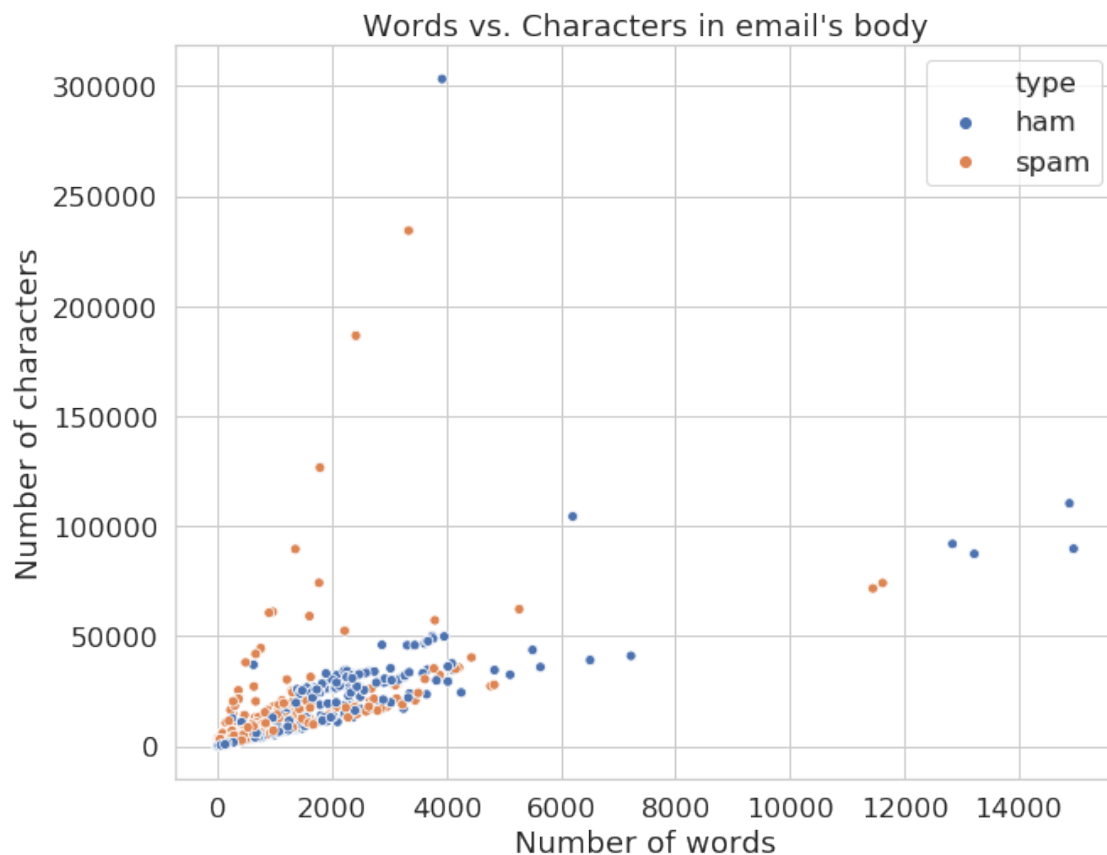
1. I went through the list of staff's suggestions, one by one. I would make visualizations to see if the suggestion would be helpful in distinguishing spam from ham emails, such as histograms or looking into its value counts. If I deemed the suggestion as helpful, I would add it as a feature to my design matrix function. For selecting certain words to use, I made a function that took in a list of words and plotted the distribution of each word and its type of email. With this, I could quickly see which words were useful in distinguishing spam from ham emails. To find better words, I randomly sampled emails and explored the email's texts to identify any pattern or recurring words. I did this for a long time, in hopes of finding better words to use.
2. I also tried to write a function to see the most common words used in spam and ham emails, but it didn't work out too well. When I kept adding words that seemed to be really good according to my proportion barplot, my validation accuracy went slightly down but my training accuracy went up. I didn't want to overfit to my training data, so I stopped there. Additionally, the word and character counts in subject titles for both spam and ham emails seemed to be similar, so I didn't use those features.
3. In my search for good features, I was surprised that separating the punctuation marks was more effective than combining all punctuation marks, which is what I did initially. '!', '@', '<' turned out to be really useful in increasing both training and validation accuracy. I was also surprised that some words you would expect to be in a spam email were not really helpful.

Generate your visualization in the cell below and provide your description in a comment.

```
In [93]: # Write your description (2-3 sentences) as a comment here:
# This is plotting the number of words against the number of characters in emails' text bodies
# Before plotting this, I initially thought these two features were related and contained some
# words does affect the number of characters. For ham emails, the linear relationship is clear
# However, looking at spam emails, a clear linear line cannot be drawn because the line can ex
# a steep slope. Therefore, one of these features cannot be excluded to clearly distinguish sp

# Write the code to generate your visualization here:
train_feature_copy = train_feature.copy()
train_feature_copy['type'] = train['spam'].apply(lambda x: "ham" if x == 0 else "spam")
plt.figure(figsize=(10,8))
sns.scatterplot(x='words_email_body', y='chars_email_body', data=train_feature_copy, hue='type')
plt.title("Words vs. Characters in email's body")
plt.xlabel("Number of words")
plt.ylabel("Number of characters")
```

```
Out[93]: Text(0, 0.5, 'Number of characters')
```



0.0.8 Question 9: ROC Curve

In most cases we won't be able to get no false positives and no false negatives, so we have to compromise. For example, in the case of cancer screenings, false negatives are comparatively worse than false positives — a false negative means that a patient might not discover a disease until it's too late to treat, while a false positive means that a patient will probably have to take another screening.

Recall that logistic regression calculates the probability that an example belongs to a certain class. Then, to classify an example we say that an email is spam if our classifier gives it ≥ 0.5 probability of being spam. However, *we can adjust that cutoff*: we can say that an email is spam only if our classifier gives it ≥ 0.7 probability of being spam, for example. This is how we can trade off false positives and false negatives.

The ROC curve shows this trade off for each possible cutoff probability. In the cell below, plot a ROC curve for your final classifier (the one you use to make predictions for Gradescope) on the training data. Refer to Lecture 19 or [Section 17.7](#) of the course text to see how to plot an ROC curve.

```
In [105]: from sklearn.metrics import roc_curve

          # Note that you'll want to use the .predict_proba(...) method for your classifier
          # instead of .predict(...) so you get probabilities, not classes

          fpr, tpr, threshold = roc_curve(Y_train, final_model.predict_proba(train_feature)[: , 1])
          plt.xlabel("False positive rate")
          plt.ylabel("True positive rate")
          plt.title("ROC curve using final classifier on training data")
          plt.plot(fpr, tpr)
          plt.show()
```

